

University of Puerto Rico

Mayagüez Campus

Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Python Performance with and without NumPy

By:

Victor A. Gerena Hilerio

Jesús Y. Cabán Feliciano

Alejandro A. Roberts Quintana

Nollan N. Rivera Febus

For: Professor Jfernando Vega Riveros

Course: ICOM 5015, Section 001D

Date: February 25, 2026

Abstract

This report evaluates the computational performance differences between pure Python implementations and NumPy-based implementations for element-wise vector multiplication and true matrix multiplication. Execution time was measured using median values over multiple trials with fixed input sizes ranging from 10 to 500 elements for vectors and 10×10 to 500×500 for matrices. All computations used float64 data types and controlled random initialization to ensure reproducibility. Results show that while performance differences for small vectors were modest, NumPy achieved substantial acceleration as problem size increased. For 500-element vectors, NumPy was approximately 24 times faster than pure Python. In matrix multiplication, the performance gap widened significantly, reaching a speedup of approximately 4303 times for 500×500 matrices. These findings demonstrate the scalability limitations of interpreted loop-based implementations and highlight the importance of compiled, vectorized numerical libraries for high-performance scientific computing.

Introduction

High-level programming languages such as Python are widely used in scientific computing and data analysis due to their reliability. However, Python's interpreted execution model introduces performance overhead, particularly when tasks are numerical and require repeated arithmetic operations [1]. To bypass these limitations, libraries like NumPy provide array-based numerical computation implemented in optimized compiled code. NumPy is expected to outperform explicit pure-Python loops, especially for matrix multiplication because its array operations are implemented in optimized compiled code paths and may dispatch linear-algebra workloads to tuned BLAS backends [2]. This project investigates the performance differences between pure Python implementations and NumPy-based implementations for two fundamental operations: element-wise vector multiplication and true matrix multiplication. By benchmarking larger inputs, this study shows how Python loops and NumPy implementations differ in performance as problem size increases. The results confirm that vectorized numerical libraries perform significantly better as problem size increases.

2. Theoretical Background

2.1 Algorithmic Complexity

The performance of matrix multiplication is fundamentally determined by its algorithmic complexity. For two square matrices of A and B of dimension $n \times n$, the classical definition of matrix multiplication is:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

The standard implementation requires n^3 multiplications and approximately n^3 additions, resulting in a computational complexity of: $O(n^3)$. This cubic growth implies that even moderate increases in matrix dimension significantly increase the number of arithmetic operations required. When implemented using triple nested loops, execution time is expected to scale proportionally to the number of elementary operations. Therefore, algorithmic complexity provides the theoretical foundation for predicting performance growth as matrix size increases.

2.2 Interpreter Overhead in Python

Python is an interpreted and dynamically typed language. In the Python implementation, each arithmetic operation involves runtime type checking, object management, reference counting, and bytecode interpretation. Unlike low-level languages, Python lists store references to objects rather than contiguous numerical values in memory.

When matrix multiplication is implemented using explicit nested loops, interpreter overhead is incurred at every iteration. Each multiplication and addition is processed individually by the interpreter, which introduces significant execution cost beyond the arithmetic operation itself. As matrix size increases, this per-operation overhead accumulates, amplifying the impact of the $O(n^3)$ growth.

Additionally, non-contiguous memory access patterns in Python lists reduce cache efficiency, further affecting computational performance.

2.3 NumPy Vectorization and Compiled Backends

NumPy improves numerical performance by using contiguous memory arrays with fixed-size homogeneous data types (e.g., float64). Unlike Python lists, NumPy arrays eliminate per-element object overhead and allow direct access to raw memory buffers.

Most importantly, NumPy delegates computationally intensive operations to optimized compiled libraries, typically implementations of Basic Linear Algebra Subprograms (BLAS). These backends are written in C or Fortran and execute arithmetic operations at native machine speed. They exploit hardware-level optimizations such as vectorized SIMD instructions, cache-aware memory access, and internal parallelization.

In this architecture, Python functions primarily serve as high-level interfaces, while the actual numerical computation is executed in compiled code. This separation significantly reduces

interpreter involvement and enables substantial performance improvements, particularly for large-scale matrix operations.

Discussion

3.1 Experimental Design

The objective of this experiment was to measure execution time differences between pure Python implementations and NumPy implementations for element-wise vector multiplication and true matrix multiplication. A fixed random seed ensured reproducibility. Arrays were created outside the timed region, and only the multiplication operation was measured. Each configuration included a warm-up execution followed by multiple trials, with the median runtime recorded. All computations used float64 data types. Correctness was verified using `np.allclose()`.

3.2 Vector Multiplication Results

Table 1 shows median execution times for vector multiplication. Performance differences increased with input size, reaching a $24.05\times$ speedup at $n = 500$.

Vector Size (n)	Median Time Python (s)	Median Time NumPy (s)	Speedup (Python/NumPy)
10.0	2e-06	1e-06	1.21
50.0	5e-06	1e-06	3.46
100.0	9e-06	1e-06	5.8
200.0	1.8e-05	2e-06	11.55
500.0	4.6e-05	2e-06	24.05

Table 1: Vector multiplication benchmark results (median values).

3.3 Matrix Multiplication Results

Table 2 presents median execution times for true matrix multiplication. Due to $O(n^3)$ scaling and interpreter overhead, pure Python runtime increased dramatically. At 500×500 , NumPy achieved a $4303.93\times$ speedup.

Matrix Size (n x n)	Median Time Python (s)	Median Time NumPy (s)	Speedup (Python/NumPy)
10.0	0.000134	3e-06	38.24
50.0	0.014649	2.1e-05	686.14

100.0	0.106929	0.000481	222.51
200.0	0.823898	0.001629	505.77
500.0	18.238976	0.004238	4303.93

Table 2: Matrix multiplication benchmark results (median values).

3.4 Comparative Performance Analysis

Figure 1 shows runtime growth on a logarithmic scale. Pure Python exhibits rapid growth consistent with $O(n^3)$ complexity and interpreter overhead. NumPy maintains significantly lower execution time due to compiled BLAS backends and optimized memory access patterns.

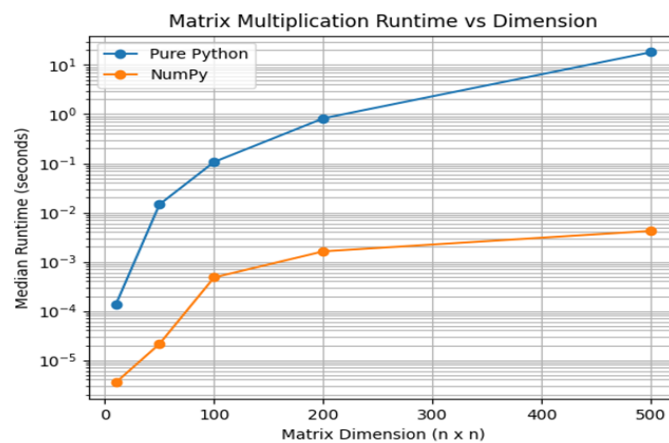


Figure 1: Matrix multiplication runtime comparison (log scale).

Figure 2 illustrates speedup as the matrix dimension increases. The widening gap demonstrates how interpreter overhead compounds with cubic growth, while NumPy leverages optimized compiled execution.

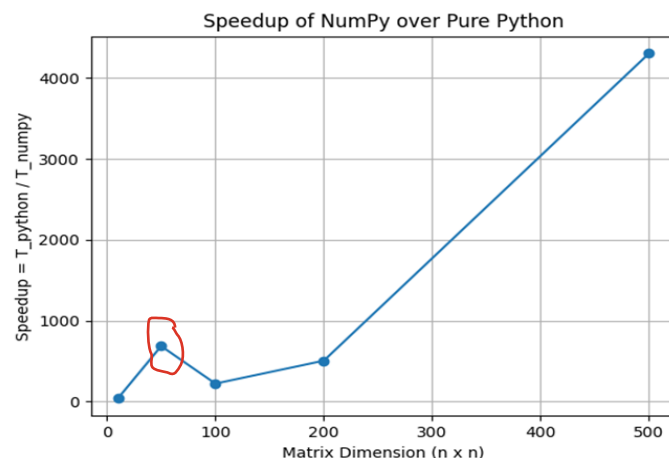


Figure 2: Speedup of NumPy over pure Python.

Conclusion

After executing the multiple element sizes of both vector and matrix multiplication, NumPy presented a consistent and efficient execution time than pure Python. During the Vector multiplication, NumPy showed a faster execution time while n increased, presenting a 24x speedup over pure Python in vector size. For matrix multiplication, while n increased, Python showed a deficient performance, largely due to the individual execution. NumPy showed a 4303.93x speedup compared to Python, proving that the raw memory buffer and compiler library optimization improved execution time, allowing large scale processing to be executed at a faster rate. While both Python and NumPy have their own unique use cases, the experiment can confirm that NumPy specialized library for numerical computing provides better performance handling large scale numerical processing, ideal for data processing and machine learning implementations.

Group Collaboration

Jesus Y. Caban: In charge of developing the code for the creation of tables, graphs, finishing the code for the elementwise and matrix multiplication, benchmarks, and discussion of results. Tested the code and confirmed the results were in accordance with what was expected

Victor A. Gerena: prepared the Abstract and Introduction sections, ensuring clarity of objectives and alignment with the theoretical framework. Additionally, created and revised the performance comparison table included in the Discussion section.

Nollan N. Rivera Febus: In charge of developing the Theoretical Background section. Researched and organized the scientific principles, models, and equations underlying the study, ensuring coherence between the theoretical framework, experimental design, and discussion of results.

Alejandro A. Roberts: Helped in creating first test versions of code with pure python and preparing the conclusion of the report. Additionally, prepared the Slides for the video presentation with the information that we gathered during the experiment.

References

- [1] Python Software Foundation, "The Python Language Reference," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/>. Accessed: Feb. 23, 2026.
- [2] NumPy Developers, "NumPy v2.0 Manual," NumPy Documentation. [Online]. Available: <https://numpy.org/doc/>. Accessed: Feb. 23, 2026.